

車牌辨識專案 學習筆記

(對照專案程式碼的問答整理，範例數字都是實際執行的結果)

相關文件：[docs/車牌辨識專案教學.pdf](#) (完整的專案與程式碼導讀)

目錄

第一章 CRNN 模型結構、super().__init__()	3
問題一：解釋一下 class CRNN	3
問題二：為什麼要 super().__init__()	5
第二章 tensor 形狀 (括號裡的數字)	7
(N, 256, 1, 32) 括號裡的數字是什麼意思？	7
第三章 LSTM、時間步與 CTC blank	10
LSTM 跟時間步是什麼？為什麼要 blank？	10
第四章 PyTorch 訓練流程	13
PyTorch 的訓練流程 (以 train.py 為例)	13
第五章 app.py 與前端的關係	18
app.py 跟前端的關係	18
第六章 @app.route 的用途	22
@app.route 的用途	22
第七章 decorator (裝飾器)	25
Decorator (裝飾器) 是什麼	25

第一章 CRNN 模型結構、super().__init__()

問題一：解釋一下 class CRNN

CRNN 定義在 plate_model.py 第 31 行，名字是 Convolutional + Recurrent Neural Network 的縮寫。它把一張車牌圖片變成「32 個時間步，每步 36 類字元的分數」，再交給 CTC 解碼成文字。整個模型分成三段：

圖片 → CNN (看圖，抽特徵) → LSTM (由左到右讀) → Linear (每步猜一個字)

1. __init__：組裝三個零件

```
def __init__(self, num_classes=len(CHARS) + 1): # 35 個字元 + 1 個 blank = 36
```

【self.cnn (第 34 行)：負責看圖】

5 個 conv_block (卷積 → BatchNorm → ReLU)，中間穿插池化。重點在池化怎麼縮小圖片：

層	輸出形狀 (通道 × 高 × 寬)	池化做了什麼
輸入	1 × 32 × 128	灰階圖
conv 64 + MaxPool(2,2)	64 × 16 × 64	高、寬都減半
conv 128 + MaxPool(2,2)	128 × 8 × 32	高、寬再減半
conv 256 × 2 + MaxPool((2,1))	256 × 4 × 32	只壓高度
conv 256 + MaxPool((4,1))	256 × 1 × 32	高度壓成 1

設計的關鍵：後面兩次池化只壓高度、不壓寬度。

- 高度最後壓成 1：字是上下方向的筆畫，資訊可以濃縮成一格。
- 寬度留下 32 格：車牌上的字是左右排列的，寬度方向的每一格對應原圖由左到右的一條窄直條。

最後的 Dropout2d(0.2) 會在訓練時隨機關掉整個通道，避免模型太依賴少數幾個特徵。

【self.rnn (第 42 行)：負責依序讀】

```
nn.LSTM(256, 128, num_layers=2, bidirectional=True, dropout=0.2)
```

- 輸入 256：每個時間步是 CNN 給的 256 維特徵。
- hidden 128 + 雙向：一個方向由左往右讀，另一個由右往左讀，各輸出 128 維，接起來是 256 維。
- 2 層：第二層 LSTM 讀第一層的輸出，能看到更複雜的前後關係。

為什麼要雙向？判斷某一格是什麼字時，左右兩邊都有幫助。例如一個字被陰影遮掉一半，旁邊幾格的資訊能幫忙補回來。

【self.fc (第 43 行) : 負責打分數】

Linear(256, 36) 把每個時間步的 256 維變成 36 個分數，分別對應 35 個字元加 1 個 blank。

2. forward : 資料怎麼流過去 (第 45 行)

```
def forward(self, x):
    f = self.cnn(x).squeeze(2)
    f = f.permute(2, 0, 1)
    out, _ = self.rnn(f)
    return self.fc(out)
```

x: (N, 1, 32, 128) · N = batch size
 # (N, 256, 1, 32) → 去掉高度那維 → (N, 256, T=32)
 # → (T, N, 256)
 # → (T, N, 256)
 # → (T, N, 36)

- squeeze(2) : 高度已經是 1 了，把這一維拿掉。
- permute(2, 0, 1) : 把「時間」換到第一維。PyTorch 的 LSTM 預設吃 (時間, batch, 特徵) 的形狀，nn.CTCLoss 也要這個順序。
- 輸出是原始分數 (logits)，還沒經過 softmax。
 訓練時 train.py 會再做 log_softmax(2) 交給 CTC loss。
 推論時 greedy_decode() 會做 softmax，取每步最大值，
 合併重複、去掉 blank，才得到文字。

3. 參數都在哪裡

訓練時印出的「模型參數量：2,351,524」，照各層形狀算一遍，總數對得上：

部分	參數量	佔比
-----	-----	-----
CNN (5 個卷積 + BatchNorm)	1,551,744	66%
LSTM (2 層雙向)	790,528	34%
Linear	9,252	0.4%

光是最後兩層 256→256 的卷積，就各有約 59 萬個參數。所以如果能把模型縮小，最有效的是減少 CNN 後段的通道數。

4. 為什麼不直接用 CNN 分類？

一般的圖片分類是「一張圖 → 一個答案」，但車牌有 6~8 個字，而且長度不固定 (AB-1234 和 ABC-1234 不一樣長)。CRNN 的做法是：

1. 固定輸出 32 個時間步，比任何車牌的字數都多。
2. 讓 CTC 用 blank 處理多出來的格子，以及「字跨好幾格」的問題。

所以訓練時只需要提供整串答案，不用標每個字在圖中的位置。這部分的細節在教學 PDF 第 5 章，那裡有一個解碼成「AB-112」的圖解例子。

問題二：為什麼要 `super().__init__()`

因為 CRNN 繼承自 `nn.Module`，而 `super().__init__()` 會先執行 `nn.Module` 自己的初始化，替你的模型準備好「登記子模組和參數的簿子」。少了這行，後面的 `self.cnn = ...` 一執行就會出錯。

實際跑過，少了這一行，模型連建立都建立不起來：

```
class Bad(nn.Module):
    def __init__(self):
        self.fc = nn.Linear(2, 2)  # 忘了 super().__init__()

Bad()
# AttributeError: cannot assign module before Module.__init__() call
```

它到底準備了什麼？

`nn.Module.__init__()` 會在物件上建立幾本內部「簿子」，最重要的是：

- `_modules`：登記子模組（例如 `cnn`、`rnn`、`fc`）
- `_parameters`：登記這一層自己的參數（權重、bias）
- `_buffers`：登記不需要訓練、但要跟著模型存檔和搬到 GPU 的數值，例如 `BatchNorm` 的平均值
- `training`：記錄現在是訓練模式還是評估模式

關鍵在於 `nn.Module` 改寫了 `__setattr__`，也就是「設定屬性」這個動作。所以當你寫：

```
self.cnn = nn.Sequential(...)
```

PyTorch 不只是把它存成一般的屬性，還會檢查「這是不是一個 `Module`」。是的話，就順手登記到 `_modules` 這本簿子裡。如果還沒呼叫 `super().__init__()`，簿子根本不存在，所以才會出現上面的錯誤。

登記之後有什麼用？

以正常的 CRNN 來看，登記好之後：

```
m._modules      → ['cnn', 'fc', 'rnn']
m.parameters()  → 38 個參數 tensor ( 就是那 235 萬個數字 )
m.state_dict()  → 'cnn.0.0.weight', 'cnn.0.0.bias', ...
```

專案裡每一個關鍵動作，都靠這份登記表才能自動往下找到所有層：

- `CRNN().to(device)`（`train.py`）
把所有層的參數一起搬到 GPU
- `torch.optim.Adam(model.parameters(), ...)`（`train.py`）
讓 optimizer 知道要更新哪些數字
- `model.train()` / `model.eval()`（`train.py`）
通知所有 Dropout、BatchNorm 切換模式

- `torch.save({"model": model.state_dict(), ...})` (`train.py`)
把所有權重存成 `plate_crnn.pt`
- `model.load_state_dict(ckpt["model"])` (`plate_model.py`)
照名字把權重放回對應的層

如果這些層沒被登記，`parameters()` 就是空的。optimizer 什麼都不會更新，存檔也是空的，模型等於永遠學不到東西。

兩個小細節

- 順序很重要：`super().__init__()` 一定要放在第一行，
在任何 `self.xxx = 某個層` 之前。
- 舊寫法：你可能會在網路上看到 `super(CRNN, self).__init__()`，
它和 `super().__init__()` 意思一樣，是 Python 2 時代的寫法。
Python 3 直接寫 `super().__init__()` 就好。

第二章 tensor 形狀 (括號裡的數字)

(N, 256, 1, 32) 括號裡的數字是什麼意思？

一句話：括號裡寫的是 tensor 的「形狀」(shape)，也就是這堆數字在每個方向 (維度) 上各有幾個。

(N, 256, 1, 32)

↑ ↑ ↑ ↑

第 3 維：寬 = 32

第 2 維：高 = 1

第 1 維：通道數 = 256

第 0 維：N = batch size (一次處理幾張圖)

(維度從 0 開始數，所以 squeeze(2) 指的是「第 2 維」= 高度。)

1. 先從熟悉的東西理解

tensor 就是「多維陣列」。形狀告訴你它長什麼樣：

(5,) 一排 5 個數字

(3, 4) 3 列 × 4 行的表格，共 12 個數字

(1, 32, 128) 一張灰階圖片：1 個通道、高 32、寬 128

(3, 32, 128) 一張彩色圖片：紅綠藍 3 個通道、高 32、寬 128

(N, 1, 32, 128) N 張灰階圖片疊在一起

元素總數 = 每個維度相乘。例如 2 張圖：

(2, 1, 32, 128) → 2 × 1 × 32 × 128 = 8192 個數字 (實際跑過，對得上)

2. 每個字母、數字代表什麼

N : batch size，一次送進模型幾張圖。

寫成 N 而不是數字，因為它會變：訓練時是 128 (或你設的 --batch)，
網頁上辨識一張圖時是 1，驗證集抽樣時是 24。

通道 (channel)：每個位置有幾個「特徵」。

輸入圖片是灰階，只有 1 個通道 (亮度)。

經過 CNN 後變成 256 個通道：每個位置有 256 個數字，

分別描述「這裡有沒有直線、有沒有圓弧、有沒有轉角.....」等特徵。

(這些特徵是模型自己學的，不是人指定的。)

高、寬：圖片 (或特徵圖) 的大小。

T：時間步 (time step) 的數量，在這個模型裡就是寬度 32。

C：類別數 (class)，這裡是 36 (35 個字元 + 1 個 blank)。

3. 實際跑一次看形狀怎麼變

拿 2 張圖 ($N = 2$) 送進 CRNN，每一步印出形狀：

步驟	形狀	說明
輸入 x	(2, 1, 32, 128)	2 張圖、1 通道、高 32、寬 128
cnn(x)	(2, 256, 1, 32)	通道變 256，高壓成 1，寬壓成 32
squeeze(2)	(2, 256, 32)	第 2 維 (高 = 1) 沒有用，拿掉
permute(2, 0, 1)	(32, 2, 256)	把維度順序換成 (時間, 圖, 特徵)
rnn	(32, 2, 256)	LSTM 讀完，形狀不變
fc	(32, 2, 36)	每個時間步 × 每張圖 → 36 個分數

這就是程式註解裡寫的：

```
f = self.cnn(x).squeeze(2)    # (N, 256, 1, 32) → 去掉高度那維 → (N, 256, T=32)
f = f.permute(2, 0, 1)        # (T, N, 256)
out, _ = self.rnn(f)          # (T, N, 256)
return self.fc(out)           # (T, N, C)
```

4. squeeze 和 permute 在做什麼

【squeeze(2)：拿掉大小為 1 的維度】

(N, 256, 1, 32) → (N, 256, 32)

高度只剩 1，就像一個只有一列的表格，那一維沒有提供任何資訊。squeeze 把它拿掉，數字一個都沒少 ($N \times 256 \times 1 \times 32 = N \times 256 \times 32$)。

【permute(2, 0, 1)：重新排列維度順序】

原本的維度編號： 0 1 2
原本的形狀： (N, 256, 32)

permute(2, 0, 1) = 新的第 0 維放原本的第 2 維 (32)
 新的第 1 維放原本的第 0 維 (N)
 新的第 2 維放原本的第 1 維 (256)

結果： (32, N, 256)

數字也一個沒少，只是換了「怎麼索引」。

為什麼要換？因為 PyTorch 的 LSTM (預設設定) 和 CTCLoss 都規定輸入要是 (時間, batch, 特徵) 這個順序。

5. 怎麼從最後的結果取數字

最後輸出的形狀是 (32, N, 36) · 用索引取值：

```
y[5, 0]      → 第 6 個時間步、第 1 張圖的 36 個分數 · 形狀 (36,)  
y[5, 0, 10]  → 上面那 36 個裡的第 11 個 · 一個數字  
y[:, 0]      → 第 1 張圖全部 32 個時間步 · 形狀 (32, 36)
```

greedy_decode() 裡的 probs.max(2) 就是「在第 2 維 (36 個類別) 裡取最大值」，得到每個時間步最可能是哪個字。

小技巧

看不懂某一步時，在程式裡加一行 `print(x.shape)` 印出形狀，大部分的困惑都會解開。PyTorch 最常見的錯誤之一就是形狀對不上（例如 `size mismatch`、`Expected 3-dimensional input`），這時候也是先把形狀印出來看。

第三章 LSTM、時間步與 CTC blank

LSTM 跟時間步是什麼？為什麼要 blank？

先看一個真實例子。用你訓練好的模型（20261002-000605）讀驗證集的000000.jpg，正確答案是 999-PRW。模型在 32 個時間步各猜了一個類別（_ 代表 blank）：

```
9 _ 9 _ 9 _ - - - - - P P _ - - - - - R _ W
```

解碼後得到 999-PRW，完全正確。下面用這一行來解釋三個概念。

1. 時間步（time step）是什麼？

「時間步」這個名字來自語音辨識：聲音是一段一段依時間播放的。在這個專案裡沒有真正的時間，它指的是「由左到右的第幾格」。

CNN 把 32×128 的圖片壓成高 1、寬 32 的特徵圖。寬度方向的 32 格，就是 32 個時間步：

```
原圖： | 9 9 9 - P R W |
        ↓ 切成 32 條窄直條，每條看原圖寬約 4 像素的區域
時間步： 1  2  3  4  ...  32
```

每個時間步是一個 256 維的向量，描述「原圖這一條窄直條長什麼樣子」。模型最後在每個時間步都要回答一次：「這一條是哪個字？還是不是字？」

所以上面那一行剛好 32 個符號，就是 32 個時間步各自的答案。

2. LSTM 是什麼？

LSTM（Long Short-Term Memory，長短期記憶網路）是一種「依序讀、而且會記住前面讀過什麼」的神經網路。

想像你用手指沿著車牌由左往右一格一格摸：

- 普通的神經網路：每一格獨立判斷，看不到旁邊。
- LSTM：手上拿著一本「筆記」，每讀一格就更新筆記，下一格判斷時會參考筆記。

LSTM 內部有三道「閘門」控制這本筆記：

- 遺忘閘（forget gate）：舊筆記哪些該擦掉
- 輸入閘（input gate）：這一格的新資訊哪些要寫進筆記
- 輸出閘（output gate）：根據筆記，這一格要輸出什麼

這些閘門也是訓練學出來的。所以模型會自己學會，例如「剛讀完一個字的右半邊，下一格大概是空白」。

為什麼這個專案用「雙向」LSTM？一個方向由左往右讀，另一個由右往左讀，最後把兩邊的筆記接起來。這樣每個時間步同時知道左邊和右邊的內容。例如某一格被陰影遮住，左右兩邊的資訊能幫忙判斷它是什麼字。

為什麼不能只用 CNN？CNN 每一格只看得到附近一小塊。LSTM 能把整串字的前後關係串起來，例如「台灣車牌的 - 前後通常是英文或數字」這種規律。

3. 為什麼要 blank？

問題在於：32 個時間步，但車牌只有 6~8 個字。一個字通常會佔好幾格，字和字之間也有空隙。所以需要規則把 32 個答案「縮」成真正的文字。CTC 的規則是：

步驟一：合併連續重複的符號
步驟二：刪掉所有 blank

用上面的例子跑一次：

```
原始：  9 _ 9 _ 9 _ - - - - - - - - - - P P _ _ _ _ _ _ _ _ R _ W
步驟一： 9 _ 9 _ 9 _ -                     P _                     R _ W
步驟二： 9  9  9  -                     P                     R  W
結果：   999-PRW
```

為什麼不能只靠「合併重複」，一定要 blank？看三個原因：

(1) 為了分開「真的重複的字」← 最關鍵的原因

答案是 999。如果沒有 blank，模型只能輸出：

```
9 9 9 9 9 9 ...
```

合併重複後只剩一個 9，永遠讀不出 999。

有了 blank，模型輸出 9 _ 9 _ 9，中間的 blank 把三個 9 隔開，合併時就不會黏在一起。

你的模型正是這樣做的：最前面的 9 _ 9 _ 9 _。

同理，車牌像 1123、AA-5566 這種有重複字的，全靠 blank。

(2) 為了表示「這一格不是字」

字和字之間的空隙、車牌的邊框、背景，這些格子不該輸出任何字。

blank 就是「這裡沒有字」的意思。

例子中 P 和 R 之間有 9 個 blank，那是兩個字之間的空白區域。

(3) 讓一個字可以佔好幾格，又不會被重複輸出

例子中 - 連續出現 12 次（連字號那一段畫得比較寬），P 出現 2 次。

合併重複後都只算一個。

這讓模型不用精準知道「字從第幾格開始、到第幾格結束」。

4. 為什麼不直接標每個字的位置就好？

可以，但非常麻煩：每張訓練圖片都得標出「第 1~3 格是 9、第 4 格是空白.....」。30000 張圖要標到天荒地老。

CTC 的厲害之處：訓練時只需要給「999-PRW」這串答案。CTC Loss 會把所有「縮完之後等於 999-PRW」的排列方式（例如 9_9_9_-P_R_W、99_9_9--PPRRW_成千上萬種）的機率加總，讓這個總機率越大越好。模型就會自己學出字大概在哪幾格。

這也解釋了為什麼訓練前 1~2 個 epoch 整張正確率是 0%：模型一開始最容易學到「全部輸出 blank」（因為大部分格子本來就是空白），要先學會字的位置，正確率才會突然往上衝。

小提醒：blank 和 - 不一樣

字元集 CHARS 裡本來就有 -（車牌上的連字號），它是真正會被輸出的字。blank 是 CTC 額外加的第 36 類，編號 0，最後會被刪掉。所以這份說明用 _ 表示 blank，避免和連字號搞混。

第四章 PyTorch 訓練流程

PyTorch 的訓練流程 (以 train.py 為例)

一句話：訓練就是不斷重複「讓模型猜 → 算猜錯多少 → 找出每個參數該往哪調 → 調一點點」，重複幾萬次，模型就越猜越準。

整體結構：

```
準備資料、模型、optimizer、loss
for 每個 epoch:                                ← 整個訓練集看一遍
    model.train()
    打亂資料順序
    for 每個 batch:                            ← 一次拿一小批圖片
        ① 前向      logits = model(x)
        ② 算 loss   loss = ctc(...)
        ③ 清梯度   optimizer.zero_grad()
        ④ 反向     loss.backward()
        ⑤ 更新     optimizer.step()
    model.eval() + 驗證，成績變好就存檔
```

下面的數字是實際跑出來的：用驗證集前 8 張圖、全新的 CRNN 示範。(只在記憶體裡跑，沒有存任何檔案。)

0. 準備：資料、模型、optimizer、loss

【資料】

```
X_train, t_train = load_split(...)          # 圖片 → tensor
targets = [torch.tensor(encode(t)) for t in t_train]
```

圖片經過 preprocess() 變成數字，答案用 encode() 變成編號：

```
X 的形狀 (8, 1, 32, 128)                    ← 8 張圖
"999-PRW" → [10, 10, 10, 35, 24, 26, 31]
      9   9   9   -   P   R   W   ( CHARS 的位置 +1 · 0 留給 blank )
```

train.py 一開始就把 30000 張全部載入記憶體，訓練時不用再讀硬碟，速度快。

【模型】

```
model = CRNN().to(device)                  # .to("cuda") 把參數搬到 GPU
```

全新的模型參數是隨機的，例如 fc 層權重的前 3 個數：

```
fc.weight[0, :3] = [0.058, 0.0613, -0.042]    grad = None ( 還沒算過梯度 )
```

【optimizer：負責「調參數」的人】

```
optimizer = torch.optim.Adam(model.parameters(), lr=a.lr)
```

`model.parameters()` 把模型全部 235 萬個參數交給它管。`lr` (learning rate) 是每次調整的步伐大小。

【scheduler：調整步伐大小】

```
scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=a.lr, ...)
```

學習率先由小升到最大，再慢慢降到很小：前期大步走得快，後期小步收得穩。

【loss function：負責「算猜錯多少」】

```
ctc = nn.CTCLoss(blank=0, zero_infinity=True)
```

1. 每個 epoch 開始

```
for epoch in range(1, a.epochs + 1):
    model.train()
    perm = torch.randperm(len(X_train))
```

- `model.train()`：切換成「訓練模式」。Dropout 會開始隨機關掉通道，BatchNorm 會用這一批的統計值並更新平均值。
- `randperm`：產生打亂的順序。每個 epoch 順序都不同，避免模型記住「第幾張後面接第幾張」這種無意義的規律。

2. 每個 batch：五個步驟（訓練的核心）

```
for s in range(0, len(perm), a.batch):
    idx = perm[s:s + a.batch]          # 這一批的編號
    x = train_augment(X_train[idx].to(device))  # 取出、搬到 GPU、即時增強
    tgt = [targets[i] for i in idx.tolist()]
```

【① 前向（forward）：讓模型猜】

```
logits = model(x)
```

實測：logits 形狀 (32, 8, 36) → 32 個時間步 × 8 張圖 × 36 類分數

`model(x)` 會自動呼叫 CRNN 的 `forward()`。

【② 算 loss：猜錯多少】

```
log_probs = logits.log_softmax(2)
loss = ctc(log_probs, torch.cat(tgt), 每張圖的時間步數, 每個答案的長度)
```

實測：全新模型的 `loss = 12.3391` (很大，因為完全在亂猜)

`loss` 是一個數字，越小代表越準。整個訓練的目標就是讓它變小。

【③ 清梯度：optimizer.zero_grad()】

PyTorch 的梯度預設會「累加」：不清掉的話，這次的梯度會加到上次上面，方向就亂了。所以每次 `backward` 之前都要先歸零。

(忘記這行是初學者最常見的 `bug` 之一，模型會訓練得很怪但不會報錯。)

【④ 反向傳播：loss.backward()】

PyTorch 自動算出「每個參數往哪個方向調、`loss` 會下降」，存在 `.grad` 裡。

實測：

```
backward 前  fc.weight.grad = None
backward 後  fc.weight.grad[0, :3] = [0.0263, 0.1308, -0.0544]
```

235 萬個參數，每個都有自己的梯度，PyTorch 全部自動算好，這就是為什麼不用自己推導微分公式。

接著 `train.py` 還有一行：

```
nn.utils.clip_grad_norm_(model.parameters(), 5)
```

梯度整體太大時等比例縮小，避免 LSTM 偶爾梯度爆炸、一步調太多把模型弄壞。

【⑤ 更新參數：optimizer.step()】

`optimizer` 根據梯度把每個參數調一點點。

實測：

```
更新前  fc.weight[0, :3] = [0.058, 0.0613, -0.042]
更新後  fc.weight[0, :3] = [0.057, 0.0603, -0.041]
```

每個都只動了約 `0.001` (= `learning rate` 的大小)。

梯度是正的就往小調，是負的就往大調，所以 `loss` 會下降。

然後 `scheduler.step()` 更新下一步要用的學習率。

3. 重複很多次之後：實測 loss 怎麼下降

拿同樣 8 張圖重複上面五步，看 `loss` 和模型讀第 1 張 (答案 999-PRW) 的結果：

步數	loss	模型讀第 1 張	8 張全對幾張
1	12.3391	'1'	0/8

```

50      3.0531  '' ( 全輸出 blank ) 0/8
100     2.0242  'W'                0/8
200     0.3503  '9PRW'              0/8
300     0.1226  '99-PRW'             3/8

```

可以看到：

- loss 穩定下降。
- 第 50 步什麼都不輸出：模型先學會「大部分格子是 blank」，這就是正式訓練前 1~2 個 epoch 正確率是 0% 的原因。
- 之後慢慢學會每個字，最後幾乎讀對。

注意：這個示範是同一批 8 張圖反覆練，等於讓模型「背答案」，只是為了看清楚流程。真正的訓練用 30000 張不同的圖，模型才能學到通用的讀字能力，而不是背起來。

4. 每個 epoch 結束：驗證和存檔

```
seq_acc, char_acc = evaluate(model, X_val, t_val, device)
```

evaluate() 裡做兩件重要的事：

```

model.eval()          # 評估模式：關掉 Dropout、BatchNorm 改用學到的平均值
with torch.no_grad():  # 不記錄計算過程、不算梯度 → 省記憶體、跑更快
    ...

```

驗證集是訓練時「沒拿來學」的 3000 張，用它打分數才知道模型是真的學會，還是只把訓練資料背起來。

成績創新高才存檔：

```

if seq_acc > best:
    torch.save({"model": model.state_dict(), "epoch": epoch, ...}, model_path)

```

state_dict() 是所有參數的「名字 → 數值」對照表，存成 plate_crnn.pt。之後 load_model() 用 load_state_dict() 把數字放回一個新建的 CRNN 裡。

5. 名詞整理

```

epoch          整個訓練集看完一遍 ( 30000 張 )
batch          一次送進模型的一小批 ( 預設 128 張 )
step / iteration 處理一個 batch ( 一個 epoch = 30000 / 128 ≈ 235 步 )
forward        資料從輸入流到輸出，得到預測
loss           預測和答案差多少的數字
gradient ( 梯度 ) 每個參數「往哪調、loss 會下降」的方向與大小
backward       自動算出所有參數的梯度
optimizer      根據梯度更新參數的演算法 ( 這裡用 Adam )
learning rate  每次更新的步伐大小
train() / eval() 切換訓練 / 評估模式 ( 影響 Dropout、BatchNorm )
no_grad()      不算梯度，只在推論、驗證時用

```

最重要的五行 (幾乎所有 PyTorch 訓練都長這樣)

```
logits = model(x)           # ① 猜
loss = loss_fn(logits, y)   # ② 算錯多少
optimizer.zero_grad()       # ③ 清掉舊梯度
loss.backward()             # ④ 算出每個參數該往哪調
optimizer.step()            # ⑤ 調一點點
```

第五章 app.py 與前端的關係

app.py 跟前端的關係

一句話：app.py 是「廚房」，前端是「點餐的客人」。前端負責畫面和互動，但它不會辨識車牌；真正跑模型的是 app.py。兩邊透過 HTTP 溝通：前端送請求（點餐），app.py 回傳 JSON（上菜）。

1. 誰是誰

後端（在你的電腦上跑的 Python 程式）

app.py Flask 網站伺服器：載入 PyTorch 模型、提供 API、
啟動訓練等背景工作

前端（在瀏覽器裡跑的東西，三個檔案）

templates/index.html	網頁的骨架：有哪些區塊、按鈕、輸入框
static/style.css	網頁的外觀：顏色、排版、進度條動畫
static/app.js	網頁的動作：按鈕按下去要做什麼、 什麼時候向 <code>app.py</code> 要資料、拿到資料怎麼顯示

為什麼要分成兩邊？

瀏覽器裡只能跑 JavaScript，跑不了 PyTorch，也讀不到你電腦上的 `models/`、`dataset/` 資料夾。所以凡是要「用模型」或「碰檔案」的事，前端都得請 `app.py` 代勞。

2. 打開網頁時發生什麼事

瀏覽器

app.py

1. 打開 `http://127.0.0.1:5000` $\longrightarrow$ `@app.route("/")` 的 `index()`
 $\longleftarrow$ 回傳 `index.html`
 (把 `{{ url_for(...)}}` 換成真正的路徑)

2. 看到 HTML 裡的 `<link>` 和 `<script>`
自動再要 `style.css`、`app.js`  Flask 內建的 `/static/` 直接回傳檔案

3. app.js 開始執行，馬上要資料：

refreshStatus()	→	GET /api/status	資料集張數、目前用哪個模型
loadHistory()	→	GET /api/history	訓練曲線的數字
loadValSamples()	→	GET /api/val-samples	驗證集抽樣辨識結果
loadModels()	→	GET /api/models	已儲存的模型清單
pollJob()	→	GET /api/jobs	背景工作狀態與執行紀錄
loadProgress()	→	GET /api/progress	訓練進度
	←	每個都回傳 JSON	

4. app.js 把 JSON 填進畫面上對應的位置 (例如 `$("modelStatus").textContent = ...`)

重點：`index.html` 一開始只是空殼，畫面上的數字、表格、圖片，全部都是 `app.js` 事後跟 `app.py` 要來再填進去的。
這也是為什麼不能直接雙擊打開 `index.html`：沒有 `app.py`，就沒人回答這些請求。

3. 幾個功能的完整流程

【上傳圖片辨識】

1. 你選了一張照片 → `app.js` 在瀏覽器裡把它畫到 `<canvas>` 上（還沒傳給後端）
2. 你拖曳框選車牌 → `app.js` 記下框的位置（換算成原圖座標）
3. 你按「辨識」→ `app.js` 用 `cropCanvas()` 在瀏覽器裡把框選範圍裁切成新圖，轉成 PNG，用 `POST /api/recognize` 上傳
4. `app.py` 的 `api_recognize()`：
讀圖 → `recognize()` → `get_model()` 取得模型 → `preprocess()` → 模型推論
→ `greedy_decode()` → 回傳 `{"text": "ABC-1234", "confidence": 0.93}`
5. `app.js` 把 `text` 和信心度顯示在畫面上

注意：裁切是在瀏覽器裡做的，後端收到的已經是車牌本身。

【隨機產生測試車牌】

1. 按按鈕 → `GET /api/random`
2. `app.py` 呼叫 `generate_dataset.py` 的 `make_sample()` 當場畫一張新車牌，辨識它，再把圖片轉成 `base64` 文字一起回傳：
`{"image": "data:image/png;base64,...", "truth": "...", "text": "...", ...}`
3. `app.js` 把 `base64` 直接塞進 `<img>` 的 `src` 顯示，並比對 `truth` 和 `text` 標出 ✓ / ✗

圖片是 `app.py` 產生的，所以要用 `base64` 夾在 `JSON` 裡送過去。

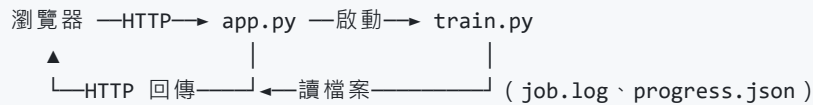
【從網頁訓練】（最複雜的一個）

1. 按「開始訓練」→ `app.js` 的 `startJob()` 送 `POST /api/jobs/train`，內容是你填的 `{epochs, batch, lr, name}`
2. `app.py` 的 `job_train()` 檢查數值範圍，再由 `start_job()` 用 `subprocess.Popen` 另外開一個 Python 程式跑 `train.py`，把它的輸出全部寫進 `logs/job.log`，然後立刻回傳 `{"ok": true}`（不會等訓練跑完，否則網頁會卡住十幾分鐘）
3. 訓練在背景跑，`train.py` 一邊訓練一邊：
 - 印文字進度到 `logs/job.log`
 - 寫 `models/runs/<時間>/progress.json`
 - 每個 epoch 更新 `history.json`、`config.json`，必要時存 `plate_crnn.pt`
4. 前端用「輪詢」追進度（每 2 秒問一次）：
 - `GET /api/jobs` → `app.py` 讀 `logs/job.log` 最後 40 行回傳 → 顯示在執行紀錄框
 - `GET /api/progress` → `app.py` 讀 `progress.json` 回傳 → 畫進度條

每 10 秒還會更新訓練曲線和模型狀態

5. 按「停止」→ POST /api/jobs/stop → app.py 用 taskkill 結束那個背景程式

三個程式的關係：



app.py 和 train.py 之間不直接對話，而是透過檔案交換資訊。

4. 為什麼用「輪詢」？

HTTP 是「前端問一次、後端答一次」的模式，後端不能主動通知前端。所以 app.js 用 setInterval 每 2 秒主動問一次「現在怎樣了？」。

```

setInterval(pollJob, 2000);           // 執行紀錄、按鈕狀態
setInterval(loadProgress, 2000);     // 進度條
setInterval(() => { loadHistory(); refreshStatus(); }, 10000); // 曲線、模型狀態
  
```

優點是簡單；缺點是有最多 2 秒的延遲，而且沒事也會一直問。（更進階的做法是 WebSocket 或 Server-Sent Events，後端可以主動推送，這個專案為了簡單沒有用。）

5. 模型怎麼在訓練中自動更新

前端完全不用管模型的事。每次辨識時，app.py 的 get_model() 會檢查：

```
key = (目前使用的模型名稱, plate_crnn.pt 的修改時間)
```

key 跟上次不同（切換了模型，或訓練中存了更好的版本）就重新載入。所以訓練時開著網站，按「隨機測試」會發現模型越來越準。

另外 recognize() 用 with lock: 包起來，確保多個請求同時進來時，不會同時載入或使用模型而互相干擾。

6. API 一覽 (前端 → app.py)

路由	方法	app.js 誰在用	做什麼
/	GET	(瀏覽器)	回傳 index.html
/api/status	GET	refreshStatus()	資料集張數、目前模型與成績
/api/models	GET	loadModels()	已儲存的模型清單
/api/models/active	POST	模型表格的「使用」按鈕	切換模型 (改寫 active.txt)
/api/recognize	POST	「辨識」按鈕	上傳圖片 → 辨識結果
/api/random	GET	「產生並辨識」按鈕	產生新車牌並辨識
/api/val-samples	GET	loadValSamples()	驗證集抽樣辨識
/api/history	GET	loadHistory()	訓練曲線資料
/api/progress	GET	loadProgress()	訓練進度
/api/jobs/generate	POST	「產生資料集」按鈕	背景產生資料集
/api/jobs/train	POST	「開始訓練」按鈕	背景訓練
/api/jobs/stop	POST	「停止」按鈕	停止背景工作
/api/jobs	GET	pollJob()	工作狀態與執行紀錄
/dataset/<split>/<檔名>	GET	驗證集抽樣的 	回傳資料集裡的圖片

錯誤處理的約定：app.py 出錯時回傳 {"error": "說明文字"} 加上狀態碼 (例如 400、404、409)。

app.js 的 getJSON() 看到狀態碼不是成功，就把 error 的文字丟成錯誤，畫面上顯示出來。例如還沒訓練模型就按辨識，會看到「尚未有訓練好的模型，請先執行 python train.py」。

7. 改程式時怎麼判斷要改哪邊

想改的東西	改哪裡
畫面上多一個按鈕、區塊	index.html (+ style.css 調外觀)
按鈕按下去的行為、顯示方式	app.js
需要用到模型、讀寫檔案、跑程式	app.py 新增一個 @app.route API， 再在 app.js 呼叫它

改完 app.py 要重開伺服器才會生效；
改完前端檔案只要在瀏覽器重新整理 (看到舊畫面就按 Ctrl + F5)。

第六章 @app.route 的用途

@app.route 的用途

一句話：@app.route 是在跟 Flask 登記「哪個網址要交給哪個函式處理」。

```
@app.route("/api/status")
def status():
    ...
    return jsonify(info)
```

意思是：「有人來要 /api/status 這個網址時，請執行 status() 這個函式，把它 return 的東西送回給瀏覽器。」

可以把 Flask 想成餐廳櫃台，@app.route 就是在櫃台的菜單上登記：「點 /api/status 的，交給 status() 這位廚師做」。

1. 前面的 @ 是什麼？

@ 是 Python 的「裝飾器」(decorator) 語法。上面那段等於：

```
def status():
    ...
status = app.route("/api/status")(status)
```

app.route(...) 會把 status 這個函式記到 Flask 內部的對照表 (app.url_map) 裡，再把函式原封不動還回來。所以它不會改變函式本身，只是「順手登記」。

2. app.py 實際登記了哪些網址

下面是直接從 app.url_map 印出來的完整對照表：

網址	方法	交給哪個函式
/	GET	index
/api/history	GET	api_history
/api/jobs	GET	job_status
/api/jobs/generate	POST	job_generate
/api/jobs/stop	POST	job_stop
/api/jobs/train	POST	job_train
/api/models	GET	api_models
/api/models/active	POST	api_set_active
/api/progress	GET	api_progress
/api/random	GET	api_random
/api/recognize	POST	api_recognize
/api/status	GET	status
/api/val-samples	GET	api_val_samples
/dataset/<split>/<filename>	GET	dataset_file

```
/static/<path:filename>      GET      static      ← Flask 自動登記，用來送 app.js、style.css
```

函式名稱不一定要和網址一樣（例如 `/api/jobs` 交給 `job_status`），Flask 只看 `@app.route` 裡寫的網址。

3. methods：限制用哪種方式請求

```
@app.route("/api/recognize", methods=["POST"])
def api_recognize():
```

- 不寫 `methods`：預設只接受 GET（「給我資料」）
- `methods=["POST"]`：只接受 POST（「我要送資料給你處理」）

這個專案的規則：

- 只是讀資料的（狀態、曲線、模型清單）→ GET
- 會上傳東西或改變狀態的（辨識上傳、開始訓練、切換模型）→ POST

用錯方法會怎樣？實際測試：

```
GET /api/recognize      → 405 Method Not Allowed (方法不對)
GET /api/nothing        → 404 Not Found (菜單上沒這道菜)
POST /api/recognize (沒附圖) → 400 {"error": "請上傳圖片"}
                                (有進到函式，是函式自己檢查後回的錯誤)
```

405 和 404 都是 Flask 自動處理的，根本不會進到你的函式。

4. 網址裡的變數：<split>/<filename>

```
@app.route("/dataset/<split>/<filename>")
def dataset_file(split, filename):
    if split not in ("train", "val"):
        return "", 404
    return send_from_directory(os.path.join(DATASET_DIR, split), filename)
```

角括號的部分是「變數」，Flask 會從網址裡切出來，當成參數傳給函式：

```
/dataset/val/000002.jpg
      ↓      ↓
split="val"  filename="000002.jpg"
```

所以一個函式就能處理成千上萬張圖片的網址。驗證集抽樣的 `` 就是靠它拿到圖片。

函式裡還檢查了 `split` 只能是 `train` 或 `val`，避免有人亂要其他資料夾的檔案。

5. 函式裡怎麼拿到前端送來的東西

三種來源，app.py 都有用到：

```
網址後面的 ? 參數 ( GET )
/api/val-samples?n=24
→ n = request.args.get("n", 24)
```

```
上傳的檔案 ( POST · 表單 )
app.js 用 FormData 附上圖片
→ file = request.files.get("image")
```

```
JSON 內容 ( POST )
app.js 送 {"epochs": 30, "batch": 128, ...}
→ b = request.get_json(force=True)
```

6. 函式要 return 什麼

return 的東西就是送回瀏覽器的內容：

```
return render_template("index.html")      → 一整個網頁 ( 只有 / 用 )
return jsonify(text=text, confidence=c)    → JSON · 給 app.js 讀
return jsonify(error="..."), 400          → JSON + 錯誤狀態碼
return send_from_directory(...)           → 一個檔案 ( 圖片 )
```

app.js 的 getJSON() 就是專門接 jsonify 回來的東西：狀態碼成功就拿資料，失敗就把 error 的文字顯示在畫面上。

7. 想新增一個功能時

例如想加一個「刪除某個模型」的功能：

1. app.py 加一個路由：

```
@app.route("/api/models/delete", methods=["POST"])
def api_delete_model():
    run = str(request.get_json(force=True).get("run", ""))
    ... ( 刪掉 models/runs/<run>/ )
    return jsonify(ok=True)
```

2. app.js 在按鈕按下時呼叫它：

```
await getJSON("/api/models/delete", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ run: r.name }),
});
```

3. 重開 app.py (新路由要重開才會登記)，瀏覽器重新整理。

(這只是示範寫法，專案裡目前沒有這個功能。)

第七章 decorator (裝飾器)

Decorator (裝飾器) 是什麼

一句話：decorator 是一個「接收函式、回傳函式」的函式，用來在不改動原函式內容的情況下，替它加上額外的行為（或順手做點事）。

寫在函式上面的 @xxx，就是在套用 decorator：

```
@timer
def slow_add(a, b):
    ...
```

完全等於：

```
def slow_add(a, b):
    ...
slow_add = timer(slow_add)    ← 把函式丟進 timer，再用回傳值取代它
```

@ 只是這行的簡寫（語法糖）。

下面每個例子都實際跑過，輸出是真的。

前提：在 Python 裡，函式也是一個「值」

函式可以像數字、字串一樣，存進變數、當參數傳、當回傳值：

```
def hello():
    return "哈囉"

f = hello          ← 注意沒有括號：這是「函式本身」，不是執行它
print(f())         ← 加括號才是執行

輸出：哈囉
```

decorator 就是建立在這個特性上：把函式當成參數傳進去，處理完再傳回來。

例 2：自己寫一個 decorator (計時器)

```
import functools, time

def timer(func):
    ← 收到原本的函式
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        ← 做一個「包裝」函式
        start = time.time()
        result = func(*args, **kwargs) ← 在中間呼叫原本的函式
        print(f"{func.__name__} 花了 {time.time() - start:.2f} 秒")
        return result
    return wrapper ← 回傳包裝後的函式

@timer
def slow_add(a, b):
    time.sleep(0.2)
    return a + b

print("結果", slow_add(1, 2))

輸出：
slow_add 花了 0.21 秒
結果 3
```

發生了什麼：

1. @timer 讓 slow_add 變成 wrapper
2. 你呼叫 slow_add(1, 2) · 其實是在呼叫 wrapper(1, 2)
3. wrapper 先記時間 → 呼叫原本的 slow_add → 印出花多久 → 回傳結果

slow_add 本身一行都沒改，就多了計時功能。如果有 10 個函式都想計時，每個前面加一行 @timer 就好。

*args, **kwargs 的意思是「不管原函式收幾個參數，都原封不動轉交過去」，這樣同一個 decorator 才能套在各種函式上。

例 3：帶參數的 decorator —— 自己做一個迷你 app.route

@app.route("/api/status") 有括號、有參數，它多了一層：

```
routes = {} ← 登記表

def route(url):
    ← 第一層：收網址
    def decorator(func):
        ← 第二層：收函式
        routes[url] = func ← 登記「這個網址 → 這個函式」
        return func ← 原封不動還回去（沒有包裝）
    return decorator

@route("/hello")
def say_hello():
    return "你好"
```

```
@route("/bye")
def say_bye():
    return "再見"

print(routes)
for url in ["/hello", "/bye", "/nothing"]:
    fn = routes.get(url)
    print(f"請求 {url} ->", fn() if fn else "404 Not Found")
```

輸出：

```
登記表: {'/hello': 'say_hello', '/bye': 'say_bye'}
請求 /hello -> 你好
請求 /bye -> 再見
請求 /nothing -> 404 Not Found
```

@route("/hello") 的執行順序：

1. 先執行 route("/hello")，得到 decorator 這個函式
2. 再執行 decorator(say_hello)，把 say_hello 登記到 routes 裡

Flask 的 @app.route 原理就是這樣，只是它的登記表叫 app.url_map，而且還會處理 methods、網址變數等等。這也是為什麼 app.py 的路由函式從來沒被你直接呼叫過：它們被登記起來，等網址請求進來時由 Flask 去呼叫。

這個例子也說明 decorator 不一定要「包裝」函式，也可以只是「順手登記」再原樣還回去。

例 4：functools.wraps 是做什麼的？

包裝之後，函式其實已經換成 wrapper 了，連名字都會變：

```
有 @functools.wraps 的 slow_add.__name__ → slow_add
沒有 wraps 的 foo.__name__ → wrapper
```

functools.wraps(func) 會把原函式的名字、說明文件等資訊複製到 wrapper 上，讓它「看起來」還是原本的函式。

在 Flask 裡這很重要：Flask 用函式名稱當作路由的識別名稱。如果你自己寫了一個沒用 wraps 的 decorator，套在兩個路由上，兩個函式都會叫 wrapper，Flask 會報錯說名稱重複（View function mapping is overwriting an existing endpoint function）。所以自己寫會包裝的 decorator 時，記得加 @functools.wraps(func)。

常見的 decorator

@app.route(...)	Flask：登記網址（這個專案裡唯一用到的 decorator）
@property	讓方法用起來像屬性：obj.area 而不是 obj.area()
@staticmethod	類別裡不需要 self 的方法
@classmethod	第一個參數是類別本身（cls）而不是物件
@torch.no_grad()	PyTorch：整個函式內不計算梯度（推論時省記憶體）
@functools.lru_cache	記住函式的結果，同樣的參數第二次直接回傳

順帶一提：這個專案的 `evaluate()` 和 `recognize()` 用的是「`with torch.no_grad():`」寫法，效果和函式上加 `@torch.no_grad()` 一樣，只是範圍限定在 `with` 區塊裡。

總結

```
@decorator      就是 func = decorator(func)
@decorator(參數) 就是 func = decorator(參數)(func)
```

decorator 常見的兩種用途：

1. 包裝：在原函式前後加行為（計時、記錄、檢查權限、快取）→ 記得用 `wraps`
2. 登記：把函式記到某個表裡，之後由別人來呼叫（Flask 的 `@app.route`）